Fingertip Gesture Control Robotic Arm

Fingertip Gesture Control Robotic Arm

1. Course Content
2. Program Startup
   - 2.1 Start the Camera and Robotic Arm Driver Environment
   - 2.2 Start the Fingertip Trajectory Robotic Arm Control Program
3. Hands-on Demonstration and Interaction Workflow
4. Core Code Analysis
   - 4.1 Node Initialization and Topic Definition
   - 4.2 Index Finger Trajectory Extraction and Geometric Contour Classification

1. Course Content

This lesson utilizes MediaPipe gesture and keypoint detection to realize contactless "fingertip trajectory drawing and shape recognition": the user only extends the index finger to draw geometric shapes in the air (such as a triangle, rectangle, circle, or five-pointed star). After drawing is completed, the user spreads the five fingers, and the system automatically recognizes the shape and directs the A1Pro robotic arm to execute the corresponding fun actions.

---

2. Program Startup

2.1 Start the Camera and Robotic Arm Driver Environment

In **Terminal 1**, enter the following command to start the robot camera and low-level driver environment:

```
ros2 launch dofbot_pro_driver A1pro_camera_arm.launch.py
```

> **Note**
>
> This all-in-one script automatically launches the image stream publishing node according to the currently installed camera model (USB monocular camera or Nuwa 3D depth camera), and loads the low-level robotic arm bus servo driver and kinematics services.

2.2 Start the Fingertip Trajectory Robotic Arm Control Program

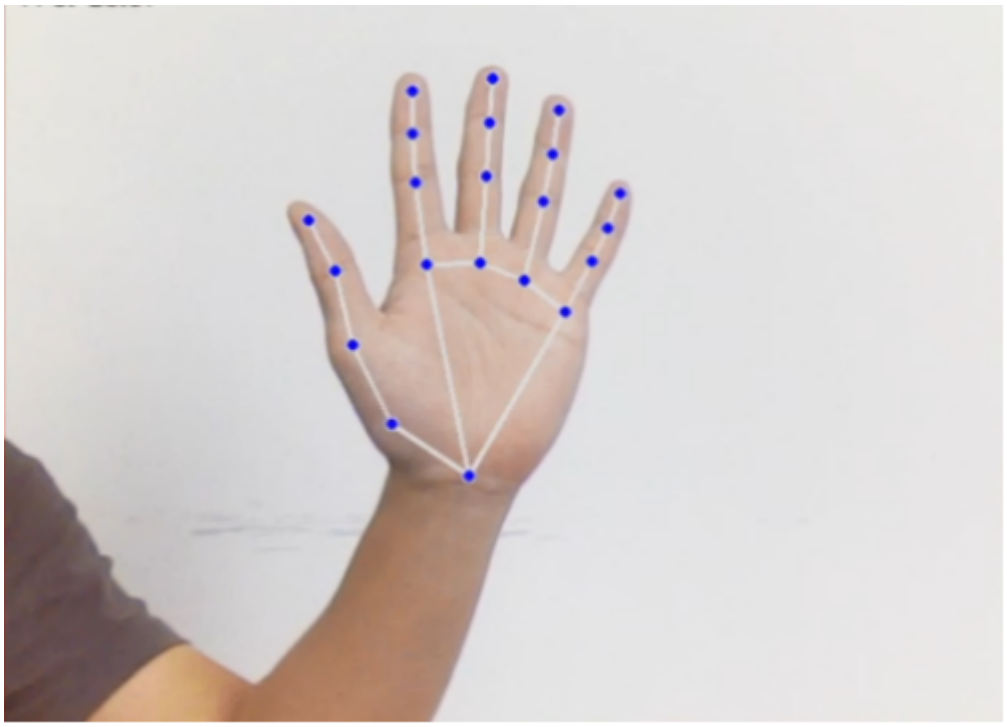
In **Terminal 2**, enter the following command to start the trajectory capture and action response node:

```
ros2 run yahboomcar_mediapipe 14_FingerAction
# Control the robotic arm to look forward
ros2 topic pub /arm6_joints dofbot_pro_interface/msg/ArmJoints "{joint1: 90, joint2: 180,
joint3: 0, joint4: 0, joint5: 90, joint6: 0, time: 1000}" --once
```

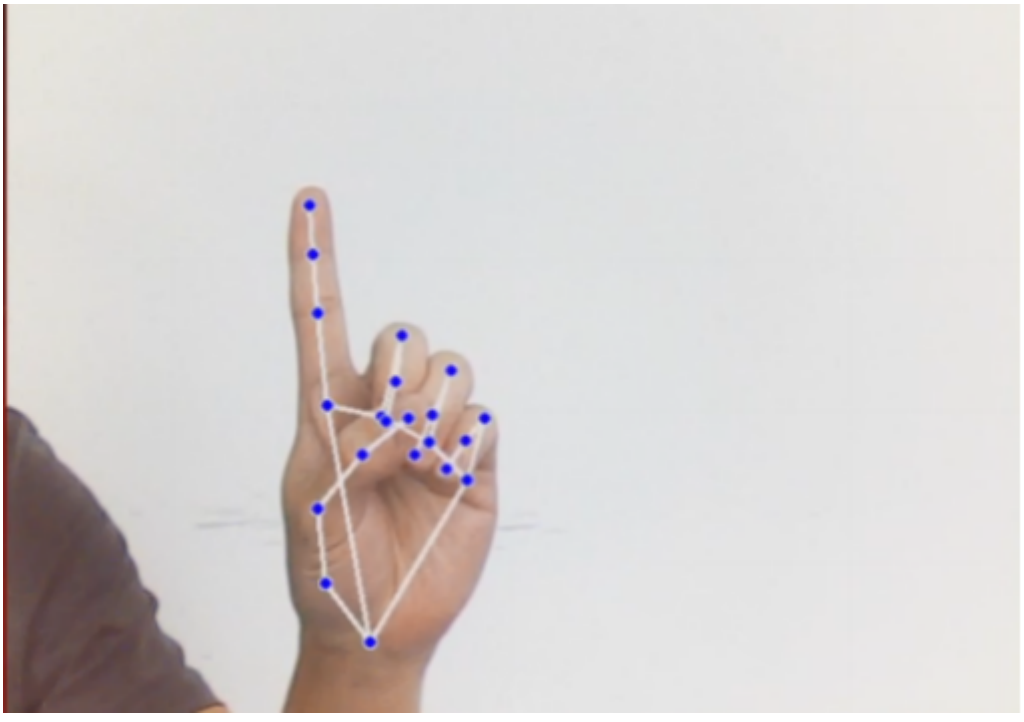
---

3. Hands-on Demonstration and Interaction Workflow

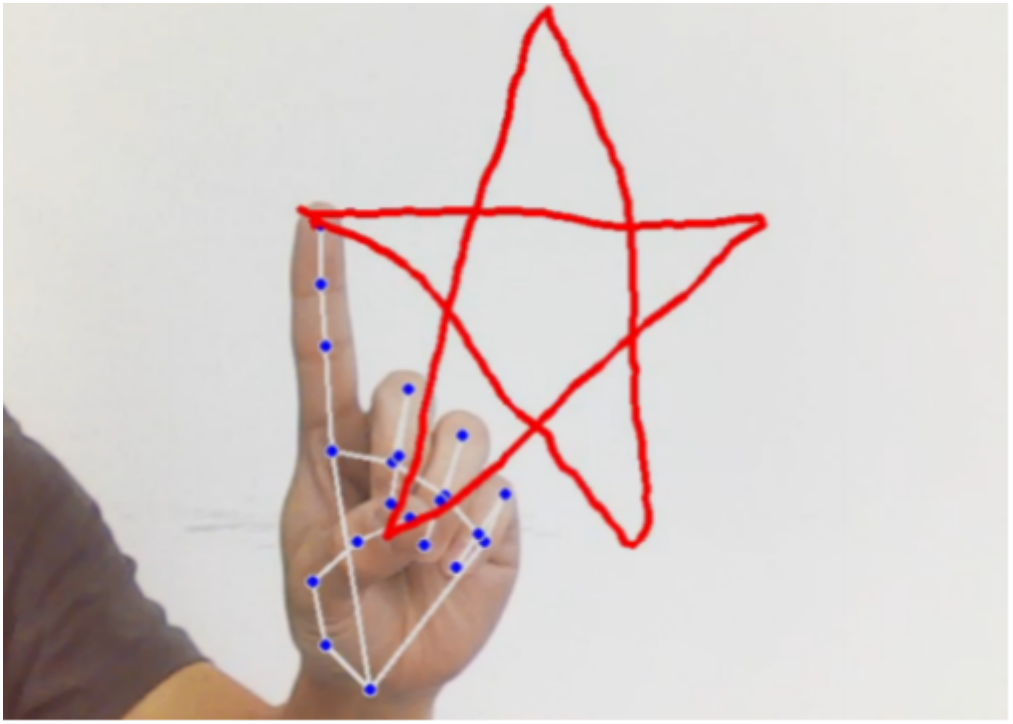
1. **Preparation and Positioning**:
    - Face the palm toward the camera with the fingers fully spread (gesture 5). The system marks the hand joints and enters the recognition preparation state;



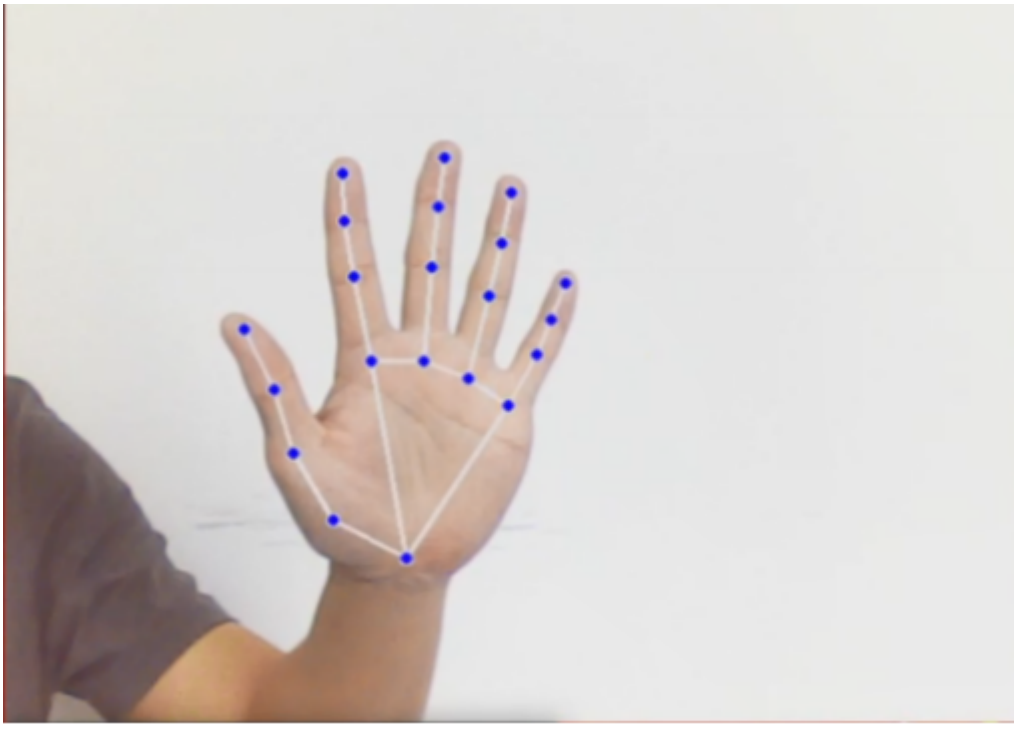
2. **Air Trajectory Drawing**:
    - Retract the other four fingers, **keeping only the index finger straightened (gesture 1)**;

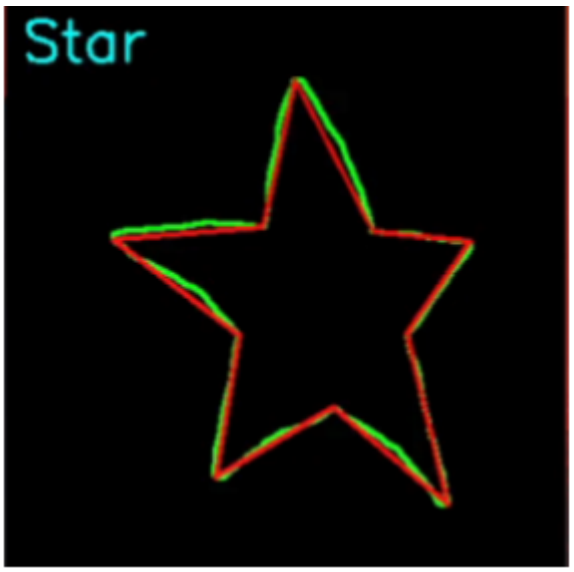


- Move the index finger slowly in the air to outline a closed shape, and the red movement trajectory will be drawn on the screen in real time;



3. **End Drawing and Action Trigger**:
    - After the shape is drawn, **quickly spread all five fingers (gesture 5)**, and the algorithm immediately locks and closes the contour and identifies the type of the drawn shape;





- The geometric shapes supported by the system include: **triangle, rectangle/square, circle, and five-pointed star**. Each shape will trigger the robotic arm to execute a different preset posture or movement action;
- Press the `Q` key in the image window or press `Ctrl+C` in the terminal to exit the program safely.

---

4. Core Code Analysis

Source code path:
`yahboomcar_ws/src/yahboomcar_mediapipe/yahboomcar_mediapipe/14_FingerAction.py`

4.1 Node Initialization and Topic Definition

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
from dofbot_pro_interface.msg import ArmJoints, ArmJoint

class FingerActionNode(Node):
    def __init__(self):
        super().__init__('finger_action_node')
        self.bridge = CvBridge()

        # Publishers for the 6-axis and single-joint control of the robotic arm
        self.pub_single_joint = self.create_publisher(ArmJoint, "arm_joint", 10)
        self.pub_six_joints = self.create_publisher(ArmJoints, "arm6_joints", 10)

        # Initial forward-looking ready posture
        self.init_joints = [90, 150, 12, 20, 90, 0]
        self.pub_six_joints.publish(self.make_arm_joints(self.init_joints))

        # Automatically subscribe to the RGB image topic
        camera_type = os.getenv('CAMERA_TYPE', 'nuwa')
        topic_name = '/ascamera_hp60c/camera_publisher/rgb0/image' if camera_type == 'nuwa'
else '/usb_cam/image_raw'
```

```
        self.sub_rgb = self.create_subscription(Image, topic_name, self.image_callback, 1)
```

4.2 Index Finger Trajectory Extraction and Geometric Contour Classification

The node tracks the pixel coordinate sequence of the index fingertip (Landmark 8) in real time, and when the five fingers are spread, it calls OpenCV's polygon approximation and corner feature analysis algorithms to complete the geometric shape classification:

```
# Fit the polygon curve and determine the shape according to the number of vertices
approx = cv2.approxPolyDP(contour, 0.04 * cv2.arcLength(contour, True), True)
vertices = len(approx)

if vertices == 3:
    shape_name = "Triangle"      # Triangle
elif vertices == 4:
    shape_name = "Rectangle"     # Rectangle
elif vertices == 5:
    shape_name = "Pentagon/Star" # Five-pointed star/Five-sided polygon
else:
    # Determine the circle according to the area/perimeter ratio (circularity)
    circularity = 4 * math.pi * area / (perimeter ** 2)
    if circularity > 0.7:
        shape_name = "Circle"
```

After the shape is recognized, the system publishes the corresponding action posture sequence to `/arm6_joints` in an independent thread, and the robotic arm automatically returns to the home position smoothly after the action is completed.